Relatório Técnico

Arquitetura DevSecOps open-source do Mentory

Integração de DAST, privacidade e SBOM no GitHub Actions

Projeto: Projeto Integrador UMC
Repositório: ProjetoIntegradorUMC/projeto-integrador
Workflow: .github/workflows/devsecops-pipeline.yml
Data da evidência: 08 de novembro de 2026

1. Introdução e contextualização

A esteira DevSecOps automatiza verificações de segurança no mesmo ciclo em que o código é alterado. Essa abordagem aplica o princípio shift-left: problemas de segurança são identificados antes da integração na branch main, quando a correção é mais rápida, barata e rastreável.

No Mentory, a automação complementa os testes funcionais e de segurança existentes em tests/test_*.py. Os testes verificam o comportamento da aplicação em execução, enquanto o workflow analisa três superfícies adicionais: segredos no histórico Git, vulnerabilidades conhecidas em dependências e padrões inseguros no código Python e JavaScript.

O workflow é acionado em push e pull_request direcionados à branch main. Cada job recebe somente contents: read, enquanto permissions: {} revoga permissões padrão do GITHUB_TOKEN. As actions externas estão fixadas por SHA completo, reduzindo o risco de alteração silenciosa de uma dependência da cadeia de suprimentos.

2. Descrição e justificativa das ferramentas

2.1 TruffleHog: detecção de segredos

O job secrets-scan usa o TruffleHog OSS para procurar credenciais, tokens e chaves expostas nas alterações e no histórico Git. O checkout usa fetch-depth: 0, portanto a ferramenta dispõe do histórico completo para investigar commits introduzidos no push ou no pull request.

```
extra_args: --only-verified
```

A opção --only-verified restringe os resultados a segredos que foram validados ativamente ou que possuem alta confiança de validade. A action também injeta o critério de falha, fazendo o job retornar erro quando encontra um segredo. Isso é mais eficaz que depender apenas do .gitignore: o .gitignore evita novos arquivos não rastreados, mas não remove credenciais já commitadas nem detecta vazamentos no histórico.

2.2 Trivy: análise de composição de software

O job sca-trivy realiza SCA em modo filesystem sobre a raiz do repositório, incluindo requirements.txt. O Trivy compara as versões declaradas com bases de vulnerabilidades públicas e produz o relatório JSON trivy-sca-report.json.

```
severity: HIGH,CRITICAL
ignore-unfixed: true
exit-code: '1'
```

HIGH e CRITICAL foram escolhidos como limiar de bloqueio porque representam risco prioritário para a integração contínua, sem transformar cada aviso de menor impacto em uma interrupção do fluxo. ignore-unfixed evita bloquear a entrega por CVEs para as quais ainda não existe versão corrigida. Quando há uma correção publicada, exit-code: '1' interrompe o job e exige tratamento antes do merge.

O binário do Trivy é baixado em versão fixa e validado por SHA-256 antes da execução. Essa validação protege a própria ferramenta usada para avaliar a cadeia de dependências.

2.3 Semgrep: análise estática de segurança

O job sast usa Semgrep 1.183.0 em ambiente virtual isolado, com os rulesets p/python, p/flask e p/javascript. A análise percorre backend/app e frontend/js sem executar a aplicação, identificando padrões associados a injeção, uso inseguro de APIs, falhas de configuração e outros problemas de código.

A escolha pelo Semgrep considera sua distribuição open-source, rulesets customizáveis e operação em repositórios públicos e privados sem depender de licença adicional. O CodeQL também seria tecnicamente adequado e possui integração semântica nativa ao GitHub, mas sua disponibilidade em repositórios privados depende do GitHub Advanced Security. No workflow atual, apenas achados ERROR bloqueiam a integração; WARNING e INFO permanecem visíveis no log informativo.

```
--severity ERROR --error --exclude tests/ --metrics off
```

3. Integração com os requisitos do projeto

A esteira não substitui os controles da aplicação. Ela fornece uma verificação contínua para os mesmos ativos e requisitos de segurança documentados no projeto.

Ferramenta	Requisito ou ativo	Integração no código	Critério de proteção
TruffleHog	Gestão de credenciais	SECRET_KEY e TOTP_ENCRYPTION_KEY em docs/autenticacao.md e docs/criptografia.md	Detectar chaves, tokens e credenciais commitados no histórico
Trivy	Dependências seguras	requirements.txt, com cryptography, Flask, bcrypt e demais pacotes fixados	Bloquear CVEs HIGH ou CRITICAL com correção disponível
Semgrep	Controles da aplicação	backend/app/routes/auth.py, lgpd.py e admin.py; frontend/js	Bloquear padrões SAST classificados como ERROR

A relação com docs/Requisitos.pdf é direta: a gestão de credenciais e a criptografia são reforçadas pela inspeção de segredos; a proteção das bibliotecas de autenticação, criptografia e persistência é reforçada pelo SCA; e os fluxos de autenticação, LGPD e auditoria são revisados continuamente pelo SAST.

- Autenticação: o arquivo docs/autenticacao.md define bcrypt, TOTP, sessões revogáveis e proteção contra enumeração.
- Criptografia: docs/criptografia.md exige que SECRET_KEY e TOTP_ENCRYPTION_KEY permaneçam fora do código e do controle de versão.
- LGPD e auditoria: os endpoints documentados em lgpd.py e admin.py permanecem no escopo da análise estática.

4. Evidências de funcionamento

As evidências abaixo foram capturadas de páginas reais do GitHub Actions, sem simulação. A execução 42532631491 ocorreu na branch main, e terminou com conclusão success.

Job	Resultado	URL do job
SAST com Semgrep	success	https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491/job/128889976248
Varredura de Segredos (TruffleHog)	success	https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491/job/128889976506
Análise de Dependências SCA (Trivy)	success	https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491/job/128889976531

A captura seguinte mostra a página da execução completa com os três jobs concluídos com sucesso. Os identificadores e a conclusão também foram conferidos pela API oficial do GitHub.

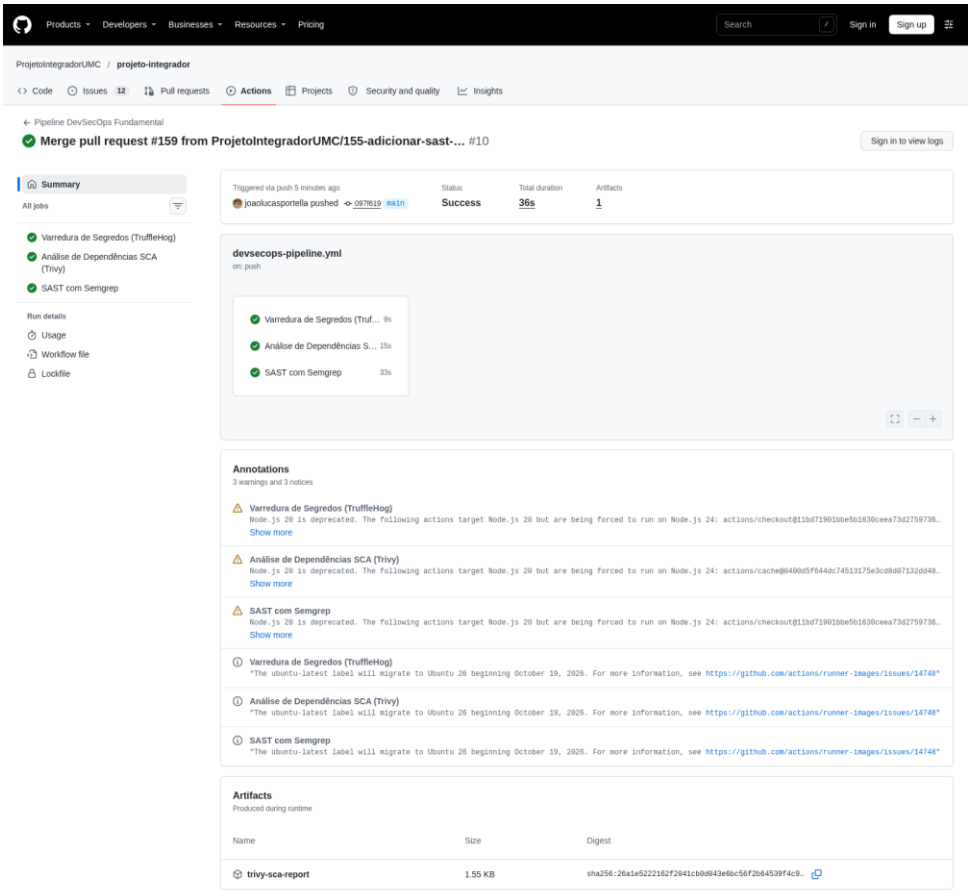


Figura 1. Execução 42532631491 com os três jobs DevSecOps concluídos com sucesso.

Fonte: https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491

O job do Trivy publicou o artefato trivy-sca-report mesmo com a condição if: always(). Na execução de referência, o artefato foi disponibilizado para download e contém um relatório JSON gerado pelo Trivy 0.69.3, com análise do requirements.txt e sem vulnerabilidades HIGH ou CRITICAL bloqueantes.

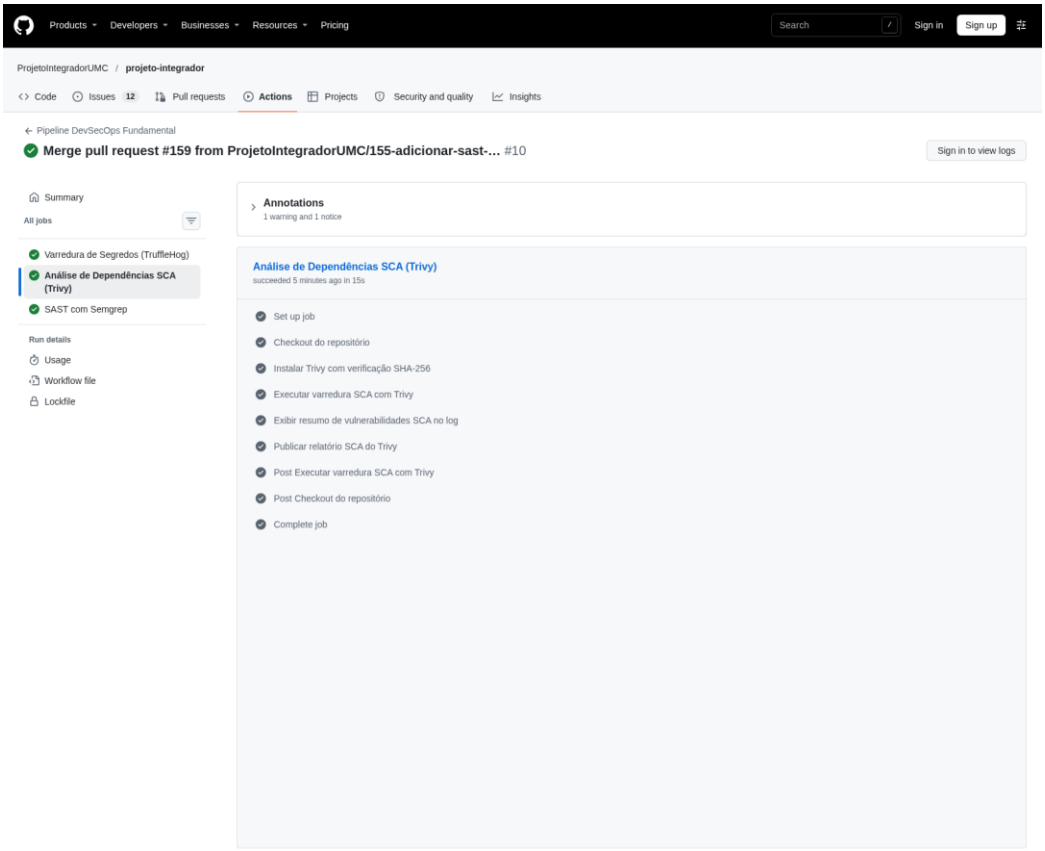


Figura 2. Job SCA do Trivy e publicação do artefato trivy-sca-report.

Artefato: https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491

Download do artefato: https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491/artifacts/13638475127

Conclusão e referências

O pipeline combina prevenção de vazamento de credenciais, análise de vulnerabilidades de dependências e análise estática de código, mantendo evidências reprodutíveis no GitHub Actions.

A execução de referência confirmou o funcionamento integrado dos três controles e a disponibilidade do relatório SCA como artefato de auditoria.

Referências técnicas

- .github/workflows/devsecops-pipeline.yml
- docs/pipeline-seguranca.md
- docs/autenticacao.md
- docs/criptografia.md
- docs/Requisitos.pdf
- requirements.txt
- https://github.com/trufflesecurity/trufflehog
- https://github.com/aquasecurity/trivy
- https://github.com/semgrep/semgrep
- https://github.com/ProjetoIntegradorUMC/projeto-integrador/actions/runs/42532631491